Chapter 12. Nonparametric Regression

Kernel regression, local polynomial regression, bandwidth and order selection, confidence intervals

Contents

12.1 Kernel regression estimation **2**

12.2 Local polynomial regression **3**

12.2.1 Choosing the degree of local polynomial regression 5

12.2.2 Choosing the bandwidth 7

12.3 Confidence intervals for the regression curve **9**

12.3.1 Theoretical pointwise intervals 10

12.3.2 Bootstrap pointwise intervals 11

Overview

Nonparametric regression extends the "linear" regression line of the linear model $Y = \beta_0 + \beta_1 X + \epsilon$ to a regression curve of free form. The nonparametric regression model is

$$Y = m(X) + \epsilon,$$

where the regression curve $m(x) = E(Y \mid X = x)$ belongs to a space $\mathcal{H}$ of smooth functions. Nonparametric regression aims to estimate $m(\cdot)$ in the very large, infinite-dimensional space $\mathcal{H}$ and to quantify its uncertainty.

Example 12.1

The results of opinion polls on the favorability of Joe Biden during his second term as vice president of the US (from 2009) are shown in Figure 12.1 of the book. To model the flow of Biden's favorability over time, the linear model cannot capture the complex change of favorability well; the red curve of Figure 12.1 captures the flow more accurately. This curve is the nonparametrically estimated regression curve.

In the data of Example 12.1 (Figure 12.1), favorability decreases around $x = 2013$ and increases in 2016. Narrowing the time range, increases and decreases alternate between 2013 and 2016: overall there is an increasing trend, but locally increases and decreases repeat. Methods using kernels to detect such local trends are the most basic tools of nonparametric regression. This chapter explains estimation of the regression curve mainly by kernel-based methods. There are also spline methods, penalized estimation, and nonparametric regression with several explanatory variables, which are not treated.

12.1 Kernel regression estimation

Assume the data $\{(x_1, y_1), \ldots, (x_n, y_n)\}$ follow the regression model

$$y_i = m(x_i) + \epsilon_i, \qquad i = 1, \ldots, n.$$

The errors ϵ_i have mean 0 and are independent. The regression curve $m(\cdot)$ is assumed continuous and smooth. Being a nonparametric model with unspecified form of the curve, estimating $m(x)$ at every x in the domain $\mathcal{X}$ of the "X" variable gives an estimate of $m(\cdot)$. Since $m(x) = E(Y \mid X = x)$, at $x = x_i$

$$m(x_i) = E(y_i),$$

so the most natural estimator of $m(x_i)$ is y_i; but a single observation varies too much, and at $x \neq x_i$ there is no corresponding y value. The kernel regression estimator at a point x uses the weighted mean of the observations near x as the estimate. Specify the width h deciding how close an observation is, and the shape of the kernel K giving higher weights to closer observations and lower to farther ones:

$$\hat{m}(x) = \sum_{i=1}^{n} \frac{K_h(x - x_i)}{\sum_{j=1}^{n} K_h(x - x_j)} y_i = \sum_{i=1}^{n} w_i^0(x)\, y_i \tag{12.1}$$

is the kernel regression estimator of $m(x)$. Here the kernel $K_h(x) = \frac{1}{h} K(\frac{x}{h})$ is the same as in kernel density estimation; the Gaussian kernel is most common. This kernel regression estimator is also called the Nadaraya–Watson estimator or "local constant" estimator. The kernel $K_h(x - x_i)$ is large when x_i is near x, so the weight $w_i^0(x)$ of each observation y_i is large only when x_i is near x.

Figure 12.2 of the book shows, with colors at the positions x_i, how the weights $w_i^0(x)$ for the dashed x vary (redder = larger weight); the blue curve is the graph of $\hat{m}(x)$ (Gaussian kernel, $h = 300$).

In kernel regression too, the choice of the width h affects the estimator more than the choice of kernel K. With the data of Example 12.1, compute the kernel regression estimate and see how it changes with h. The R code below uses data `x`, `y`, divides the domain of "X" into 100 points `xgrid`, and defines `kernreg()` computing the kernel regression estimate at each; `Kh()` computes the kernel $K_h(x)$.[1]

```
x = as.numeric(poll$start_date %>% as_date(), "days")
y = poll$Favorable
xgrid = round(seq(min(x),max(x), length.out = 100))
Kh = function(x,h){ exp( - x^2 /h^2)/ sqrt(2*pi)/h}
kernreg = function(xgrid, x, y, h){
  kx = sapply(x,function(xi){ Kh(xgrid-xi,h)})
  W <- kx / rowSums(kx)
  drop(W %*% y)
}
```

(The Gaussian kernel is $\exp(-x^2/(2h^2))/(\sqrt{2\pi}h)$; the book's `Kh()` omits the factor 2 in the exponent, which amounts to using width $h/\sqrt{2}$, and the normalization cancels in the weights.) With width $h = 100$ (days), `mhat = kernreg(xgrid,x,y,100)` computes $\hat{m}(x)$. Figure 12.3 of the book shows the estimates for $h = 10, 100, 300$: a small width gives a very rough curve, a large width a smoother curve. What happens to $\hat{m}(\cdot)$ if h is made infinitely large ($h \to \infty$)?[2]

12.2 Local polynomial regression

The kernel regression estimator is a special case of the local polynomial regression estimator. First see why it is called a "local constant" estimator. As least squares is the basic estimator in linear regression, the kernel regression estimator (12.1) can be written

$$\hat{m}(x) = \arg\min_{\alpha} \sum_{i=1}^{n} (y_i - \alpha)^2 w_i^0(x) = \arg\min_{\alpha} \sum_{i=1}^{n} (y_i - \alpha)^2 K_h(x - x_i). \tag{12.2}$$

(Supplied: setting the derivative in α to zero gives $\sum_i (y_i - \alpha) K_h(x - x_i) = 0$, i.e. $\alpha = \sum_i w_i^0(x) y_i$.) The kernel regression estimate $\hat{m}(x)$ at x estimates α in the constant model $y_i = \alpha + \epsilon_i$ using only the observations around it; hence it is an estimator using a locally constant model. Local polynomial regression replaces the constant model of (12.2) by a polynomial model such as $y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \epsilon_i$. In principle any pth-degree polynomial ($p = 0, 1, 2, \dots$) can be used; for simplicity take a first-degree polynomial. Replacing the constant α of (12.2) by $\alpha + \beta x$,

$$(\hat{\alpha}_x, \hat{\beta}_x) = \arg\min_{\alpha,\beta} \sum_{i=1}^{n} (y_i - \alpha - \beta x_i)^2 K_h(x - x_i). \tag{12.3}$$

That is, the linear regression line fitted locally "around" x is $y = \hat{\alpha}_x + \hat{\beta}_x x$, and the prediction at x of this linear model is the local polynomial regression estimate

$$\hat{m}_1(x) = \hat{\alpha}_x + \hat{\beta}_x x. \tag{12.4}$$

(The subscript "1" indicates a first-degree polynomial.) As the position x changes, $(\hat{\alpha}_x, \hat{\beta}_x)$ changes, so $\hat{m}_1(\cdot)$ is a curve of free form like the kernel regression curve. The local polynomial estimator $\hat{m}_1(\cdot)$ (12.4) has the form of local linear regression (12.3) and is called the **local linear regression estimator**.

[1] `np_supp.R` defines `lcreg()`, similar to `kernreg()`; for the name `lcreg()` see the next section.

[2] If h is infinitely large, the kernel regression estimate is constant for all x: $\hat{m}(x) \equiv \bar{y}$. (Supplied, Exercise 12.1: as $h \to \infty$, $K_h(x - x_i)/K_h(x - x_j) = K((x - x_i)/h)/K((x - x_j)/h) \to K(0)/K(0) = 1$ for continuous K with $K(0) > 0$, so all weights tend to $1/n$. So the simple regression line is not a special case of the Nadaraya–Watson estimator.)

Figure 12.4 of the book shows, at two different positions x, the graph of the locally fitted regression line and the value of $\hat{m}_1(x)$. The local linear estimator $\hat{m}_1$ is the collection of the results of locally fitted regression lines.

The computation of local polynomial estimators, including local linear regression, is weighted least squares. The general form of (12.3) at a given x,

$$\arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i - \cdots - \beta_p x_i^p)^2 K_h(x - x_i), \tag{12.5}$$

can be written in matrix form. With

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \qquad X = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^p \\ 1 & x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{pmatrix}, \qquad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix},$$

and $W_x = \operatorname{diag}\{K_h(x - x_1), \ldots, K_h(x - x_n)\}$, the $n \times n$ diagonal matrix with ith diagonal element $K_h(x - x_i)$, (12.5) becomes

$$\hat{\boldsymbol{\beta}}_x = \arg\min_{\boldsymbol{\beta}} (\mathbf{y} - X\boldsymbol{\beta})' W_x (\mathbf{y} - X\boldsymbol{\beta}) = (X' W_x X)^{-1} X' W_x \mathbf{y}. \tag{12.6}$$

Proof of (12.6) (supplied; Exercise 12.2). $Q(\boldsymbol{\beta}) = (\mathbf{y} - X\boldsymbol{\beta})' W_x (\mathbf{y} - X\boldsymbol{\beta})$ is a convex quadratic (W_x is positive semidefinite). Its gradient is $-2X' W_x (\mathbf{y} - X\boldsymbol{\beta})$, and setting it to 0 gives the normal equations $X' W_x X \boldsymbol{\beta} = X' W_x \mathbf{y}$. When $X' W_x X$ is invertible (at least $p + 1$ distinct x_i with positive weight), the unique minimizer is $(X' W_x X)^{-1} X' W_x \mathbf{y}$. ■

So the local polynomial estimate $\hat{m}_p(x)$ at x, with $\mathbf{x}_x = (1, x, x^2, \ldots, x^p)'$, is

$$\hat{m}_p(x) = \mathbf{x}_x' \hat{\boldsymbol{\beta}}_x = \begin{pmatrix} 1 & x & \cdots & x^p \end{pmatrix} \hat{\boldsymbol{\beta}}_x = \mathbf{x}_x' (X' W_x X)^{-1} X' W_x \mathbf{y} = \sum_{i=1}^{n} w_i^p(x)\, y_i, \tag{12.7}$$

where the weight $w_i^p(x)$ of pth-degree local polynomial regression is the ith element of the $1 \times n$ row vector $\mathbf{x}_x' (X' W_x X)^{-1} X' W_x$.[3]

The local polynomial estimate $\hat{m}_p(x)$ with $p = 0$ is the Nadaraya–Watson estimator; with $p = 1$ it is the local linear estimator; for $p \geq 2$ generally the pth-degree local polynomial estimator. The weights of the local linear estimator are

$$w_i^1(x) = \frac{b_i(x)}{\sum_{j=1}^{n} b_j(x)}, \qquad b_i(x) = \{S_2(x) + (x - x_i) S_1(x)\} K_h(x - x_i), \tag{12.8}$$

where $S_q(x) = \sum_{j=1}^{n} K_h(x - x_j)(x_j - x)^q$ for $q = 0, 1, 2, \ldots$.

Weights for $p = 0, 1$ (supplied; Exercise 12.3). For $p = 0$, $X = \mathbf{1}$ and $\mathbf{x}_x = 1$, so $w_i^0(x) = (\mathbf{1}' W_x \mathbf{1})^{-1} K_h(x - x_i) = K_h(x - x_i) / \sum_j K_h(x - x_j)$, as in (12.1). For $p = 1$, the fitted value at x is unchanged if we center the design at x, i.e. use columns $(1, x_i - x)$; then $\mathbf{x}_x = (1, 0)'$ and $\hat{m}_1(x)$ is the intercept. With $k_i = K_h(x - x_i)$ and $d_i = x_i - x$, $X' W_x X = \begin{pmatrix} S_0 & S_1 \\ S_1 & S_2 \end{pmatrix}$, and the first row of its inverse is $\frac{1}{S_0 S_2 - S_1^2}(S_2, -S_1)$. So

$$\hat{m}_1(x) = \frac{\sum_i (S_2 - S_1 d_i) k_i y_i}{S_0 S_2 - S_1^2},$$

and $\sum_i (S_2 - S_1 d_i) k_i = S_2 S_0 - S_1^2$. With $b_i = (S_2 - S_1 d_i) k_i = \{S_2 + (x - x_i) S_1\} K_h(x - x_i)$ this is (12.8). ■

[3] `np_supp.R` defines `lcreg()` (local constant regression) and `llreg()` (local linear regression) for $p = 0, 1$; with them the weights $w_i^p(x)$ can be examined.

Example 12.2

The local polynomial estimate $\hat{m}_p(x)$ can be computed with `locpoly()` of the R package `KernSmooth`. For Biden's favorability data, set the degree p with `degree` and the width h with `bandwidth`:

```
> p = 1
> h = 50
> ll = KernSmooth::locpoly(x,y,degree = p, bandwidth = h)
> head(as.data.frame(ll),3)
         x        y
1 15276.00 41.00002
2 15280.94 40.97017
3 15285.88 40.93969
```

The output of `locpoly()` is the pairs $(x, \hat{m}_p(x))$ on a grid of x. `llreg()` defined in `np_supp.R` also computes local linear estimates:

```
> xgrid = ll$x
> llregout = llreg(xgrid, x, y, h)
> cbind(xgrid, llregout$mhat) %>% head(3)
        xgrid
[1,] 15276.00 41.00000
[2,] 15280.94 41.01326
[3,] 15285.88 41.02651
```

The estimates at each x differ slightly because `llreg()` implements the definition (12.7) of the local linear estimate exactly but computes slowly, while `KernSmooth::locpoly()` gives an approximation of $\hat{m}_1(x)$ for fast computation.

The local polynomial estimate is determined by the degree p of the local polynomial and the kernel width h. Which degree and width should be chosen? The next section answers.

12.2.1 Choosing the degree of local polynomial regression

Figure 12.5 of the book shows the local polynomial estimates computed as in Example 12.2 for each of $p = 0, 1, 2$ and $h = 50, 300$. At the left and right ends of the horizontal axis the differences between the three estimates are clear. With the kernel width fixed, a higher degree gives a more complex estimated curve. At the right end of the horizontal axis, a higher degree follows the increasing pattern of the scatter plot more faithfully. That is, the higher the degree of the polynomial, the smaller the bias of the estimator:

$$\text{bias}\{\hat{m}_0(x)\} > \text{bias}\{\hat{m}_1(x)\} > \text{bias}\{\hat{m}_2(x)\} > \cdots . \tag{12.9}$$

By the bias–variance trade-off one would predict the variance grows with the degree. This is not exactly true, however. At the left end of the horizontal axis in Figure 12.5, the variability of the estimated curves for $p = 0, 1$ is similar, while that for $p = 2$ is much larger. The hypothesis suggested is

$$\text{Var}\{\hat{m}_0(x)\} = \text{Var}\{\hat{m}_1(x)\} < \text{Var}\{\hat{m}_2(x)\} = \text{Var}\{\hat{m}_3(x)\} < \cdots : \tag{12.10}$$

the variance increases with the degree, but the variance of a local polynomial estimator of odd degree equals that at the even degree one lower.

For details, consider only $p = 0, 1$. The Nadaraya–Watson estimator for $p = 0$,

$$\hat{m}_0(x) = \sum_{i=1}^{n} w_i^0(x)\, y_i, \tag{12.11}$$

and the local linear estimator for $p = 1$,

$$\hat{m}_1(x) = \sum_{i=1}^{n} w_i^1(x)\, y_i, \tag{12.12}$$

are both linear combinations of the y_i, and the bias and variance of the estimate $\hat{m}_p(x)$ at x are closely tied to the weights $w_i^p(x)$.

Theorem 12.1

In the model $Y_i = m(x_i) + \sigma(x_i)\epsilon_i$, $\sigma(x_i) > 0$, with the ϵ_i independent with mean 0 and variance 1, the bias and variance of the Nadaraya–Watson estimator (12.11) and the local linear estimator (12.12) are, for $p = 0, 1$,

$$\text{bias}[\hat{m}_p(x)] = \sum_{i=1}^{n} w_i^p(x)\{m(x_i) - m(x)\}, \tag{12.13}$$

$$\text{Var}[\hat{m}_p(x)] = \sum_{i=1}^{n} \{w_i^p(x)\, \sigma(x_i)\}^2. \tag{12.14}$$

Proof (supplied). The weights satisfy $\sum_i w_i^p(x) = 1$ (for $p = 0$ by definition; for $p = 1$ by (12.8)). Since the weights depend only on the x's, $E\hat{m}_p(x) = \sum_i w_i^p(x) m(x_i)$, and subtracting $m(x) = \sum_i w_i^p(x) m(x)$ gives (12.13). The Y_i are independent with $\text{Var}\, Y_i = \sigma^2(x_i)$, giving (12.14). ■

Theorem 12.1 assumed the error variances may differ. To express the bias and variance more simply asymptotically, we need: first, for the term $m(x_i) - m(x)$ of the bias to be simply expressed by Taylor expansion, the regression curve $m(\cdot)$ must be smooth. Second, not only the response Y_i but also the explanatory variable X_i must be random, so that the weighted means containing X_i in (12.13) and (12.14) are approximated by expectations.

To express the asymptotic bias and variance of the Nadaraya–Watson and local linear estimators, assume the following. We estimate the regression curve $m(x)$ at $x \in \mathcal{X}$ in some interval $\mathcal{X} = (a, b)$ of the explanatory variable.

Assumption A. $m(\cdot)$ is smooth on $\mathcal{X}$.[4]
Assumption B. The density $f(\cdot)$ of X_i is smooth and $f(x) > 0$ for $x \in \mathcal{X}$.

In the model

$$Y_i = m(X_i) + \sigma(X_i)\epsilon_i, \qquad i = 1, \ldots, n,$$

the bias and variance of $\hat{m}_0(x)$ and $\hat{m}_1(x)$ conditional on the observed explanatory values $\mathbb{X}_n = (X_1, \ldots, X_n)$ are:

Theorem 12.2 (García-Portugués (2021), Theorem 4.1)

Let $\hat{m}_0(x)$, $\hat{m}_1(x)$ be the Nadaraya–Watson and local linear estimators with the Gaussian kernel. For $p = 0, 1$, if Assumptions A and B hold and $n \to \infty$, $h \to 0$, $nh \to \infty$,

$$\text{bias}[\hat{m}_p(x) \mid \mathbb{X}_n] = B_p(x)h^2 + o_P(h^2), \tag{12.15}$$

$$\text{Var}[\hat{m}_p(x) \mid \mathbb{X}_n] = \frac{\sigma^2(x)}{2\sqrt{\pi} f(x)} \frac{1}{nh} + o_P((nh)^{-1}), \tag{12.16}$$

[4] More precisely, the second derivative $m''(\cdot)$ exists and is continuous.

where

$$B_p(x) := \begin{cases} \dfrac{m''(x)}{2} + \dfrac{m'(x)f'(x)}{f(x)}, & p = 0, \\ \dfrac{m''(x)}{2}, & p = 1. \end{cases}$$

Sketch (supplied). For $p = 0$, $\hat{m}_0(x) - m(x) = \frac{\frac{1}{n}\sum_i K_h(x-X_i)\{m(X_i)-m(x)+\sigma(X_i)\epsilon_i\}}{\frac{1}{n}\sum_i K_h(x-X_i)}$. The denominator converges in probability to $f(x)$. For the bias part of the numerator, with $X_i = x - hu$, $E[K_h(x-X)\{m(X)-m(x)\}] = \int K(u)\{m(x-hu)-m(x)\}f(x-hu)\,du$; expanding $m(x-hu)-m(x) = -hum' + \frac{h^2u^2}{2}m'' + o(h^2)$ and $f(x-hu) = f - huf' + o(h)$ and using $\int uK = 0$, $\int u^2K = 1$ gives $h^2\{\frac{m''f}{2} + m'f'\} + o(h^2)$; dividing by f gives B_0h^2. For the variance, $\frac{1}{n^2}\sum_i K_h^2(x-X_i)\sigma^2(X_i) \approx \frac{\sigma^2(x)f(x)}{nh}\int K^2$, and $\int K^2 = \frac{1}{2\sqrt{\pi}}$ for the Gaussian kernel; dividing by $f(x)^2$ gives (12.16). For $p = 1$ the local linear fit removes the linear term exactly: the weights satisfy $\sum_i w_i^1(x)(X_i - x) = 0$ (the fit reproduces lines). So the m' contribution, including the cross term $m'f'/f$ produced by the asymmetry of the design density, cancels and only $\frac{m''}{2}h^2$ remains, while the leading variance is unchanged. ■

From Theorem 12.2, for the local polynomial estimators with $p = 0, 1$:

1. The bias (12.15) is proportional to the square of the kernel width h; the variance (12.16) is inversely proportional to nh. Both hold for $p = 0, 1$ and are the same as the asymptotic properties of the kernel density estimator. With the sample size n fixed, a larger width increases the bias but reduces the variance, and a smaller width increases the variance but reduces the bias: the bias–variance trade-off.
2. The local linear estimator has smaller bias than the Nadaraya–Watson estimator. More specifically, the term $\frac{m'(x)f'(x)}{f(x)}$ present in the bias for $p = 0$ disappears for $p = 1$. The term $\frac{m'(x)f'(x)}{f(x)}$ present only in the bias of the Nadaraya–Watson estimator is related to the distribution $f(x)$ of the explanatory variable and is also called the "design bias."[5] (This confirms roughly the hypothesis (12.9) for $p = 0, 1$.)
3. The first term of the bias, $m''(x)$ (the second derivative of $m(x)$), is larger the more sharply the regression curve $m(x)$ changes.
   - If $m''(x) = 0$, the slope of the curve $m(\cdot)$ does not change at x (it is locally linear), and the bias of the local linear estimator $\hat{m}_1(x)$ is 0.
   - If $m''(x) > 0$, the curve $m(\cdot)$ forms a "valley" near x; then $E[\hat{m}_1(x)] > m(x)$: the estimate tends to lie above the valley floor.
   - If $m''(x) < 0$, the curve forms a "peak" near x; then $E[\hat{m}_1(x)] < m(x)$: the estimate tends to lie below the peak.
4. The first term of the variance (12.16) is the same for $p = 0$ and $p = 1$. (This confirms roughly the hypothesis (12.10) for $p = 0, 1$.)

Compared with the Nadaraya–Watson estimator ($p = 0$), the local linear estimator ($p = 1$) has (asymptotically) the same variance and smaller bias. So local linear regression is generally preferred to Nadaraya–Watson. Likewise local cubic is preferred to local quadratic. For simplicity, from now on only the local linear estimator is used.

12.2.2 Choosing the bandwidth

For local linear estimation of the regression curve $m(\cdot)$, a good estimator is one whose overall estimated curve $\hat{m}_1$ has small MISE, and the MISE is closely tied to the kernel width h. Integrating

[5] Recall that in multiple regression the matrix X of the explanatory variables was called the design matrix.

only the first two terms of Theorem 12.2 gives the approximation

$$\mathrm{MISE}(\hat{m}_1(X) \mid \mathbb{X}_n) \approx h^4 \int \frac{(m''(x))^2}{4} f(x)\,dx + \frac{\int \sigma^2(x)\,dx}{2\sqrt{\pi}} \frac{1}{nh}. \tag{12.17}$$

Regarding the approximated MISE (12.17) as a function of h, the value of h minimizing it is the asymptotically optimal width h^*:

$$h^* = \Big(\frac{\int \sigma^2(x)\,dx}{2\sqrt{\pi} \int (m''(x))^2 f(x)\,dx} \cdot \frac{1}{n} \Big)^{\frac{1}{5}}. \tag{12.18}$$

(Supplied: here the MISE weights the squared bias by the design density, $\int \{\cdot\}^2 f(x)\,dx$, while the variance term $\frac{\sigma^2(x)}{2\sqrt{\pi} f(x) nh}$ integrated against f gives $\frac{\int \sigma^2}{2\sqrt{\pi} nh}$; minimizing $Ah^4 + \frac{B}{nh}$ with $A = \frac{1}{4}\int (m'')^2 f$ and $B = \frac{\int \sigma^2}{2\sqrt{\pi}}$ gives $h^5 = \frac{B}{4An}$, i.e. (12.18).) The form of the denominator of (12.18) is very similar to (11.10) of Section 11.2.3. As with the optimal width in kernel density estimation, (12.18) cannot actually be used. Of the methods estimating the optimal width h^* directly, the following plug-in method is representative.

Plug-in method. The expression (12.18) of the optimal width h^* contains the second derivative $m''(x)$ of the regression curve, the marginal distribution $f(x)$ of the explanatory variable, and the error variance $\sigma^2(x)$ that can vary with position. Ruppert, Sheather and Wand (Ruppert et al., 1995) proposed estimating $\int (m''(x))^2 f(x)\,dx$ and $\int \sigma^2(x)\,dx$ directly. Roughly: divide the range $\mathcal{X}$ of the explanatory variable into N blocks $\mathcal{X}_1, \ldots, \mathcal{X}_N$; in each block estimate the regression curve as a quartic polynomial using only that block's data, call it $\tilde{m}(\cdot)$, and write the second derivative of that quartic as $\tilde{m}''(\cdot)$. Then

$$\int (m''(x))^2 f(x)\,dx = E[(m''(X))^2] \approx \frac{1}{n} \sum_{i=1}^{n} (\tilde{m}''(X_i))^2 =: \tilde{m}^2.$$

Also, from the residual sums of squares of the quartic regressions in the blocks, obtain error variance estimates $\tilde{\sigma}_1^2, \ldots, \tilde{\sigma}_N^2$ and approximate $\int \sigma^2(x)\,dx$ by their weighted average:

$$\int \sigma^2(x)\,dx \approx \sum_{j=1}^{N} \tilde{\sigma}_j^2 |\mathcal{X}_j| =: \tilde{\sigma}^2.$$

Choosing the number of blocks N by a model selection criterion such as AIC or BIC, the plug-in kernel width is

$$h^{\mathrm{DPI}} = \Big(\frac{\tilde{\sigma}^2}{2\sqrt{\pi} \tilde{m}^2} \cdot \frac{1}{n} \Big)^{\frac{1}{5}}.$$

The Ruppert–Sheather–Wand "direct plug-in" method is implemented in R as `KernSmooth::dpill` (direct plug-in for local linear regression).

Example 12.3

For Biden's favorability data, setting the width with the Ruppert–Sheather–Wand direct plug-in method gives $h^{\mathrm{DPI}} = 57.79$:

```
> (h.dpi = KernSmooth::dpill(x,y))
[1] 57.78665
```

Cross-validation. The second way of choosing the kernel width is cross-validation, often used to examine prediction performance in regression. Given $\hat{m}_1(\cdot)$ estimated from the data $\mathbb{D}$, if a new observation (X', Y') arrived, how close would $\hat{m}_1(X')$ be to Y'? To measure this we need an observation (X', Y') independent of the data $\mathbb{D}$ (or of $\hat{m}_1$ estimated from it). Leave-one-out cross-validation generates this situation from the data: for each $i = 1, \ldots, n$, from the data without the ith observation,

$$\mathbb{D}_{(-i)} = \{(x_1, y_1), \ldots, (x_n, y_n)\} \setminus \{(x_i, y_i)\},$$

compute the local linear estimate $\hat{m}_1^{(-i)}(\cdot)$, and obtain the prediction error $y_i - \hat{m}_1^{(-i)}(x_i)$ of the held-out (x_i, y_i). For a given kernel width h the mean prediction error is

$$\mathrm{CV}(h) = \frac{1}{n} \sum_{i=1}^{n} \{y_i - \hat{m}_1^{(-i)}(x_i)\}^2, \tag{12.19}$$

and the width minimizing it, $h^{\mathrm{cv}} = \arg\min_{h>0} \mathrm{CV}(h)$, is the cross-validated kernel width. As in leave-one-out validation in linear regression, computing (12.19) does not require n local linear fits: writing $\hat{m}_1(x) = \sum_{i=1}^{n} w_i^1(x) y_i$ as in (12.12), it is known that

$$\mathrm{CV}(h) = \frac{1}{n} \sum_{i=1}^{n} \left\{ \frac{y_i - \hat{m}_1(x_i)}{1 - w_i^1(x_i)} \right\}^2.$$

The leave-one-out shortcut (supplied). Removing observation i from the weighted least squares at $x = x_i$: the leave-one-out fit equals the full fit with y_i replaced by its own leave-one-out prediction $\tilde{y}_i = \hat{m}_1^{(-i)}(x_i)$, because adding a point that lies exactly on the fitted local line leaves that fit unchanged. Hence $\tilde{y}_i = \sum_{j \neq i} w_j^1(x_i) y_j + w_i^1(x_i) \tilde{y}_i$, i.e. $\tilde{y}_i = \frac{\hat{m}_1(x_i) - w_i^1(x_i) y_i}{1 - w_i^1(x_i)}$. Therefore $y_i - \tilde{y}_i = \frac{y_i - \hat{m}_1(x_i)}{1 - w_i^1(x_i)}$. ■

Example 12.4

For Biden's favorability data, compute $\mathrm{CV}(h)$ for candidate widths $h \in \{40, 42, 44, \ldots, 120\}$:

```
> h.range = seq(40, 120, by = 2)
> K = length(h.range)
> cv = vector(length = K)
> for (k in 1:K){
+   h = h.range[k]
+   llreg.out = llreg(x,x,y,h)
+   cv[k] = mean( ( (y - llreg.out$mhat) /
+                   (1- diag(llreg.out$weight)) )^2 )
+ }
> (h.cv = h.range[which.min(cv)])
[1] 70
```

$\mathrm{CV}(\cdot)$ is smallest at $h = 70$ (plot in the book). With the cross-validated width $h^{\mathrm{cv}} = 70$ and the plug-in width $h^{\mathrm{DPI}} = 57.79$ of Example 12.3, the local linear estimates are compared in Figure 12.6 of the book; the two widths are almost the same, so the estimates nearly coincide.

12.3 Confidence intervals for the regression curve

In a nonparametric model of very free form, an exact confidence interval for the regression curve is very hard to obtain. As in density estimation, with the optimal width h^* the standard error and bias of the local linear estimator are of the same size,[6] so the bias in the estimator is hard to remove.

[6] When $h^* \asymp n^{-1/5}$, both standard error and bias are of size $n^{-2/5}$. (By comparison, in simple linear regression the standard error of the least squares estimator is $n^{-1/2}$ and the bias 0.)

As in density estimation, intervals are given not for the actual regression function $m(x)$ but for the mollified regression curve

$$m_h(x) = E(\hat{m}_1(x)),$$

where h is the kernel width of $\hat{m}_1(x)$. For x in an interval $\mathcal{X} = (a, b)$ of the explanatory variable, pointwise intervals for $m_h(x)$ are obtained asymptotically and by the bootstrap. Simultaneous bands are rarely used in practice (Hall and Horowitz, 2013) and are not explained.

12.3.1 Theoretical pointwise intervals

The local linear estimator $\hat{m}_1(x)$ is a weighted mean of the responses y_i as in (12.12), so by the CLT it is asymptotically normal. Using the expression of Theorem 12.1,

$$\frac{\hat{m}_1(x) - m_h(x)}{\mathrm{se}(\hat{m}_1(x))} \dot{\sim} N(0, 1), \tag{12.20}$$

where $\mathrm{se}^2(\hat{m}_1(x)) = \mathrm{Var}(\hat{m}_1(x)) = \sum_{i=1}^n \{w_i^1(x)\sigma(x)\}^2$, and the error variance $\sigma^2(x)$ is unknown and must be estimated.

- **(Homoscedasticity)** If $\sigma^2(x) = \sigma^2$, as in linear models σ^2 is estimated by dividing the residual sum of squares by its "degrees of freedom." With the response vector $\mathbf{y} = (y_1, \ldots, y_n)'$, the "hat matrix" that computes its fitted values is the $n \times n$ matrix $W = (w_{ij})_{i,j=1,\ldots,n}$, $w_{ij} = w_j^1(x_i)$, so that $\hat{\mathbf{y}} = W\mathbf{y}$. (As in (12.12), check that $\hat{y}_i := \hat{m}_1(x_i) = \sum_{j=1}^n w_j^1(x_i) y_j$.) The residual vector of local linear regression is

  $$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = (I - W)\mathbf{y},$$

  and the consistent estimator of the error variance σ^2 is[7]

  $$\hat{\sigma}^2 = \frac{\mathbf{y}'(I - W)'(I - W)\mathbf{y}}{\mathrm{trace}((I - W)'(I - W))} = \frac{\sum_{i=1}^n (y_i - \hat{m}_1(x_i))^2}{\mathrm{trace}((I - W)'(I - W))} \tag{12.21}$$

  (Wasserman, 2006, Theorem 5.85).
- **(Heteroscedasticity)** Suppose $\sigma^2(x)$ varies with x. Assuming the variance function $\sigma^2(\cdot)$ is smooth, estimate $\sigma^2(\cdot)$ by smoothing the bivariate data (x_i, r_i^2) containing the squared residuals $r_i^2 = (y_i - \hat{m}_1(x_i))^2$. With the method of Yu and Jones (2004), the estimated variance function can be forced to be always positive: with $Z_i = \log r_i^2$ as response and x_i as explanatory variable, nonparametric regression gives an estimate $\hat{q}(x)$ of the regression curve, and

  $$\hat{\sigma}^2(x) = \exp(\hat{q}(x)).$$

Once the error variance is estimated, substituting

$$\widehat{\mathrm{se}}(\hat{m}_1(x)) = \Big(\sum_{i=1}^n \{w_i^1(x)\hat{\sigma}(x)\}^2 \Big)^{1/2} \tag{12.22}$$

into (12.20) gives the (asymptotic) $1 - \alpha$ interval for $m_h(x)$:

$$m_h(x) \in \hat{m}_1(x) \pm z_{1-\alpha/2} \widehat{\mathrm{se}}(\hat{m}_1(x)).$$

This interval corresponds to the plug-in large-sample interval of density estimation, but differs in estimating the standard error directly rather than through a "plug-in" of an asymptotic expression.

[7] For an $n \times n$ matrix $A = (a_{ij})$, $\mathrm{trace}(A) = \sum_{i=1}^n a_{ii}$ is the sum of the diagonal elements of A.

Example 12.5

Intervals for the local linear estimate with kernel width $h^{\text{cv}} = 70$ as in Example 12.4. Assuming homoscedastic errors, $\hat{\sigma}^2$ is computed by (12.21):

```
> n = length(x)
> llreg.x = llreg(x, x, y, h = h.cv)
> residuals = y - llreg.x$mhat
> tr = sum( diag( t(diag(n) - llreg.x$weight) %*% (diag(n) -
    llreg.x$weight)))
> (hat.sigma2 = sum(residuals^2) / tr)
[1] 12.09957
```

At each point x of the grid of the "X" variable where intervals are wanted, compute the regression estimate $\hat{m}_1(x)$ and its standard error (12.22), then the theoretical interval:

```
> xgrid = seq(min(x), max(x), length.out = 100)
> llreg.out = llreg(xgrid, x, y, h = h.cv)
> mhat = llreg.out$mhat
> se = sqrt( rowSums(llreg.out$weight^2) * hat.sigma2 )
> ci.theory = data.frame(x = xgrid,
+                        ci.low = mhat - qnorm(1-0.025)*se,
+                        mhat = mhat,
+                        ci.up = mhat + qnorm(1-0.025)*se)
```

These intervals are shown in the first panel of Figure 12.7 of the book.

12.3.2 Bootstrap pointwise intervals

Nonparametric regression is also regression, so two kinds of bootstrap resampling are possible: the case bootstrap and the residual bootstrap. With the kernel width fixed at h, the bootstrap replicate of the local linear estimate $\hat{m}_1(\cdot)$ is $\hat{m}_1^*(\cdot)$, computed from a bootstrap sample with the same width, and is easily obtained by case-bootstrap resampling. In nonparametric regression the estimator always has bias, which is also reflected in the residuals. Residual-bootstrap methods removing the effect of this bias have been developed, but exceed the level of this book; we use only the case bootstrap.

The nonparametric bootstrap resampling of the local linear estimate:

1. From the original bivariate data $\mathbb{D} = \{(x_i, y_i) : i = 1, \ldots, n\}$ estimate the kernel width h^* and with it compute the local linear estimate $\hat{m}_1(\cdot)$.
2. Draw n observations (x_i, y_i) with replacement from $\mathbb{D}$ to get $\mathbb{D}^* = \{(x_i^*, y_i^*) : i = 1, \ldots, n\}$.
3. From the bootstrap sample $\mathbb{D}^*$ compute the local linear estimate $\hat{m}_1^*(\cdot)$ with width h^*.
4. Repeat steps 2–3 B times to get the bootstrap replicates $\hat{m}_1^{*b}(\cdot)$, $b = 1, \ldots, B$.

With these replicates, estimate the standard error directly and substitute into (12.20) for the bootstrap plug-in interval, or use the percentile bootstrap interval.

Bootstrap plug-in interval. At each $x \in \mathcal{X}$ the bootstrap-estimated standard error is

$$\widehat{\text{se}}^{\text{Boot}}(\hat{m}_1(x)) = \Big(\frac{1}{B-1}\sum_{b=1}^{B}(\hat{m}_1^{*b}(x) - \overline{\hat{m}_1^*(x)})^2\Big)^{1/2},$$

and the $1-\alpha$ bootstrap plug-in interval is

$$m_h(x) \in \hat{m}_1(x) \pm z_{1-\alpha/2}\widehat{\text{se}}^{\text{Boot}}(\hat{m}_1(x)).$$

Percentile interval. With $m^*(x;\alpha/2)$ and $m^*(x;1-\alpha/2)$ the $\alpha/2$- and $(1-\alpha/2)$-quantiles of the bootstrap replicates $\hat{m}_1^{*1}(x),\ldots,\hat{m}_1^{*B}(x)$ at each $x\in\mathcal{X}$, the $1-\alpha$ bootstrap percentile interval for $m_h(x)$ is

$$m_h(x)\in\big(m^*(x;\alpha/2),\ m^*(x;1-\alpha/2)\big).$$

Example 12.6

Both the bootstrap plug-in and percentile intervals need only the bootstrap replicates of $\hat{m}_1(x)$. The R code below generates $B=1999$ of them:

```
xgrid = seq(min(x), max(x), length.out = 100)
mhat = llreg(xgrid, x, y, h = h.cv)$mhat
set.seed(1)
B = 1999
m.boot = replicate(B,{
boot.id = sample(1:n,n,replace = TRUE)
x.boot = x[boot.id]
y.boot = y[boot.id]
llreg(xgrid, x.boot, y.boot, h = h.cv)$mhat
}) %>% t()
```

With a grid $(x_{(1)},\ldots,x_{(N)})$ of length N (here $N=100$), the stored `m.boot` is a $B\times N$ matrix whose (b,j) element is $\hat{m}_1^{*b}(x_{(j)})$. For the 95% bootstrap plug-in interval compute the bootstrap standard error estimate of $\hat{m}_1(x)$:

```
se.boot = apply(m.boot,2,sd)
ci.pi = data.frame(x = xgrid,
                   ci.low = mhat - qnorm(1-0.025)*se.boot,
                   mhat = mhat,
                   ci.up = mhat + qnorm(1-0.025)*se.boot)
```

The 95% percentile interval is given by the 2.5% and 97.5% quantiles of each column of `m.boot`:

```
q1 = apply(m.boot,2,function (x) {quantile(x,0.025)})
q2 = apply(m.boot,2,function (x) {quantile(x,1-0.025)})
ci.perc = data.frame(x = xgrid,
                     ci.low =q1,
                     mhat = mhat,
                     ci.up = q2)
```

The bootstrap intervals are shown in the second and third panels of Figure 12.7 of the book.